

从提示词到多智能体框架： AutoGen、CrewAI 及其核心思想

计算机专业学习笔记

2026 年 6 月 2 日

目录

1 为什么要从“直接聊天”升级到“智能体框架”？

1.1 最直观的比喻：一个员工 vs. 一个团队

假设你需要完成一项复杂任务：计算一个积分，并查询水的分子量。你可以：

- **雇一个全能员工**，让他凭记忆或当场学习解决所有问题——这就像直接调用 DeepSeek 的聊天 API，每次都在提示词里告诉他“你现在是数学家，也是化学家”。
- **组建一个团队**，你作为老板把任务交给主管，主管自己找数学专家和化学专家分工，最后汇总报告——这就是多智能体框架（如 AutoGen、CrewAI）在干的事。

显然，第一种方式在任务简单时完全够用；但任务一复杂、涉及多个专业领域、需要调用外部工具时，你就得自己当“传令官”在多个 API 调用之间来回传话，很容易出错。框架帮你把传令的过程自动化了。

1.2 从 API 调用到 Agent 循环：一个典型场景的演化

1. **原始聊天模式**：用户输入 → 模型输出。无法使用工具，无法自主决策。

2. **手动 Function Calling 循环**：程序员需要实现一个循环：

```
1 while True:
2     response = client.chat.completions.create(messages=history, tools=tools)
3     if response.choices[0].message.tool_calls:
4         # 手动解析、执行工具、把结果塞回 history
5         execute_and_update_history()
6     else:
7         break
8
```

每次添加新工具或改变协作模式，都要重写这部分胶水代码。

3. **多智能体框架**：框架把这个循环封装成 Agent 的内部机制，并提供 Agent 间通信、任务分发、并行执行等高级功能。你只需要定义角色和工具。

2 核心概念：Agent、Tool、Orchestration

2.1 Agent（智能体）是什么？

在框架语境下，一个 Agent 不仅仅是“一个带有系统提示词的模型”。它通常具有：

- **身份与目标**（通过 system message 定义）
- **记忆**（对话历史，可能包含长期记忆）
- **工具集**（能调用的函数、API、甚至其他 Agent）
- **决策循环**（根据环境反馈决定下一步行动，例如思考、调用工具、回复用户）

2.2 Tool（工具）与 Function Calling

大模型本身不能执行代码、访问网络或查询数据库。**Function Calling** 允许模型输出一个“函数调用请求”（包含函数名和参数），由外部系统实际执行，再将结果返回给模型。框架将这一过程标准化为 **Tool** 的注册和调用。

生动理解：餐厅点菜

想象你在餐厅点菜。你（用户）告诉服务员（Agent）你要什么。服务员不能自己做菜，他写下菜单（Function Call）传到后厨（工具执行环境）。厨子做好后，服务员把菜端回给你。如果没有这个流程，你就得自己对厨子喊，中间很容易漏单。框架就是这个餐厅的管理系统，确保服务员和厨师高效配合。

2.3 多智能体编排（Multi-Agent Orchestration）

当一个 Agent 无法完成任务时，它可以把子任务委派给其他 Agent。编排模式常见有：

- **层级式**：一个主管 Agent 将任务分派给专家 Agent，并汇总结果（如我们运行的多智能体示例）。
- **顺序式**：Agent A 的输出成为 Agent B 的输入，像流水线。
- **群聊式**：所有 Agent 在一个聊天室里，各自发表意见，最后得出结论。
- **辩论式**：多个 Agent 就一个观点进行辩论，以得到更严谨的答案。

3 AutoGen 框架探秘

3.1 分层架构

AutoGen 的设计是分层的，每一层解决不同的问题：

1. **Core API**：最底层，提供消息传递、事件驱动机制，支持本地和分布式运行时，甚至允许 Python 和 .NET Agent 互操作。

2. **AgentChat API**: 构建在 Core 之上, 提供更简单的抽象, 封装了单 Agent、双 Agent 对话、群聊等常用模式, 是我们使用的那层。
3. **Extensions API**: 允许接入各种模型客户端 (OpenAI、Azure、DeepSeek 等)、代码执行器、MCP 工具等。

这种架构让你可以从快速原型逐渐深入到需要精细控制的分布式系统。

3.2 我们遇到的“非 OpenAI 模型适配”问题

在部署中, 我们使用了 DeepSeek (通过 OpenAI 兼容接口)。直接使用 `OpenAIChatCompletionClient(model=)` 会报错, 因为 AutoGen 无法识别非 OpenAI 的模型名。解决办法是提供 `ModelInfo` 对象, 手动声明模型的能力 (如是否支持视觉、函数调用、JSON 输出等)。这体现了框架的**模型无关设计**——通过抽象保证灵活性, 但要求开发者对模型能力有清晰认识。

```
1 model_client = OpenAIChatCompletionClient(  
2     model="deepseek-chat",  
3     base_url="https://api.deepseek.com/v1",  
4     api_key=os.getenv("DEEPSEEK_API_KEY"),  
5     model_info=ModelInfo(  
6         vision=False,  
7         function_calling=True,  
8         json_output=True,  
9         family="unknown",  
10        structured_output=True,  
11    ),  
12 )
```

Listing 1: 适配 DeepSeek 的模型客户端配置

3.3 多 Agent 协作实例分析

我们运行了以下逻辑:

- 定义 `math_expert` 和 `chemistry_expert` 两个 Agent。
- 通过 `AgentTool` 将它们封装成主 Agent 可调用的工具。
- 主 Agent 收到任务后, 经过“思考”, 决定是直接回答还是调用专家工具。

结果显示: 对于简单的积分问题, 主 Agent 自己回答了; 对于水的分子量, 它自动调用了化学专家。这个过程没有一行手动的工具调用代码, 框架完成了整个 loop。

4 其他主流多智能体框架对比

4.1 CrewAI

CrewAI 采用“角色扮演”的方式, 定义 Agent、Task、Crew, 并指定一个执行过程 (process)。语法更接近自然语言, 非常适合快速搭建原型。底层也基于 LangChain, 因此生态丰富。如果说 AutoGen 像严谨的企业 ERP 系统, CrewAI 就像用便利贴组建的敏捷小组。

4.2 LangGraph

LangGraph (来自 LangChain) 不是一个纯粹的多智能体框架，而是一个**有状态的图计算框架**，适合构建自定义的 Agent 工作流。你可以用节点表示 Agent 或工具，用边表示控制流，并实现复杂的条件分支和循环。它更底层，灵活性最高，但需要更多代码。

4.3 OpenAI Swarm

OpenAI 推出的轻量级多智能体实验框架，核心思想是“交接” (handoffs): 一个 Agent 可以直接将对话控制权移交给另一个 Agent。设计极简，适合教学和概念验证。

如何举一反三？

不论框架名字多花哨，其本质都是 **Agent + 工具 + 通信协议**。掌握了：

- **Agent 循环**：观察 → 思考 → 行动 → 观察的闭环。
- **工具抽象**：统一的调用接口和结果反馈。
- **消息传递**：Agent 之间如何交换信息（文本、结构化消息、事件）。

你就能快速上手任何新框架。比如，当看到 CrewAI 的 Task 对象时，你会明白它不过是把一个 Agent 调用包装成了带输入输出的任务；看到 AutoGen 的 GroupChat，不过是把所有 Agent 的消息广播到一个房间。

5 实战经验：在 Fedora 上部署 AutoGen 的收获

1. **环境隔离**：Python 虚拟环境是必须的，避免依赖冲突。
2. **模型适配**：除了 OpenAI, 很多模型都支持兼容接口，但需要显式声明能力 (如 ModelInfo), 否则框架无法正确路由工具调用。
3. **工具依赖管理**：MCP 示例虽然因系统浏览器问题没有跑通，但让我们理解了工具集成可能涉及系统级依赖 (浏览器、图形库)。实际项目中应考虑容器化 (Docker) 来保证环境一致性。
4. **参数版本敏感**：框架更新快，官方文档的示例代码可能和当前版本不完全匹配 (如 AgentTool 不再接受 `return_value_as_last_message`)。此时直接查看源码帮助 (`help()`) 是最可靠的方式。

6 总结：从玩玩具到建工厂

- **提示词工程**是“教一个人怎么做”，而 **Agent 框架**是“建立一套制度让人自动做”。
- 多智能体框架的价值在于**自动化协调**，让你从繁琐的胶水代码中解放出来，聚焦业务逻辑。

- 选择框架时，先看生态和社区活跃度，再看抽象层级：想要快速验证用 CrewAI；需要精细控制和分布式部署用 AutoGen；想完全自定义流程用 LangGraph。
- 永远不要被框架的术语吓到——它们无非是把“调用模型、处理工具、传递消息”这三件事做了不同程度的封装。

掌握本质，方能举一反三。